University of Exeter

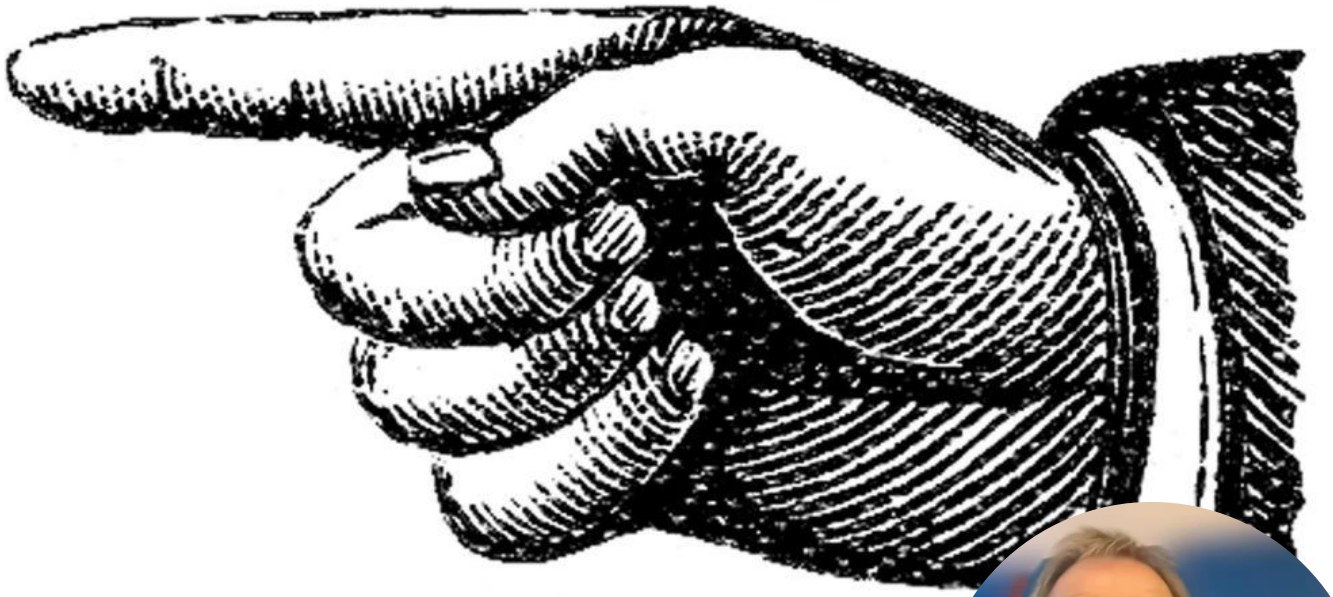
ECM2433: The C Family

Pointers

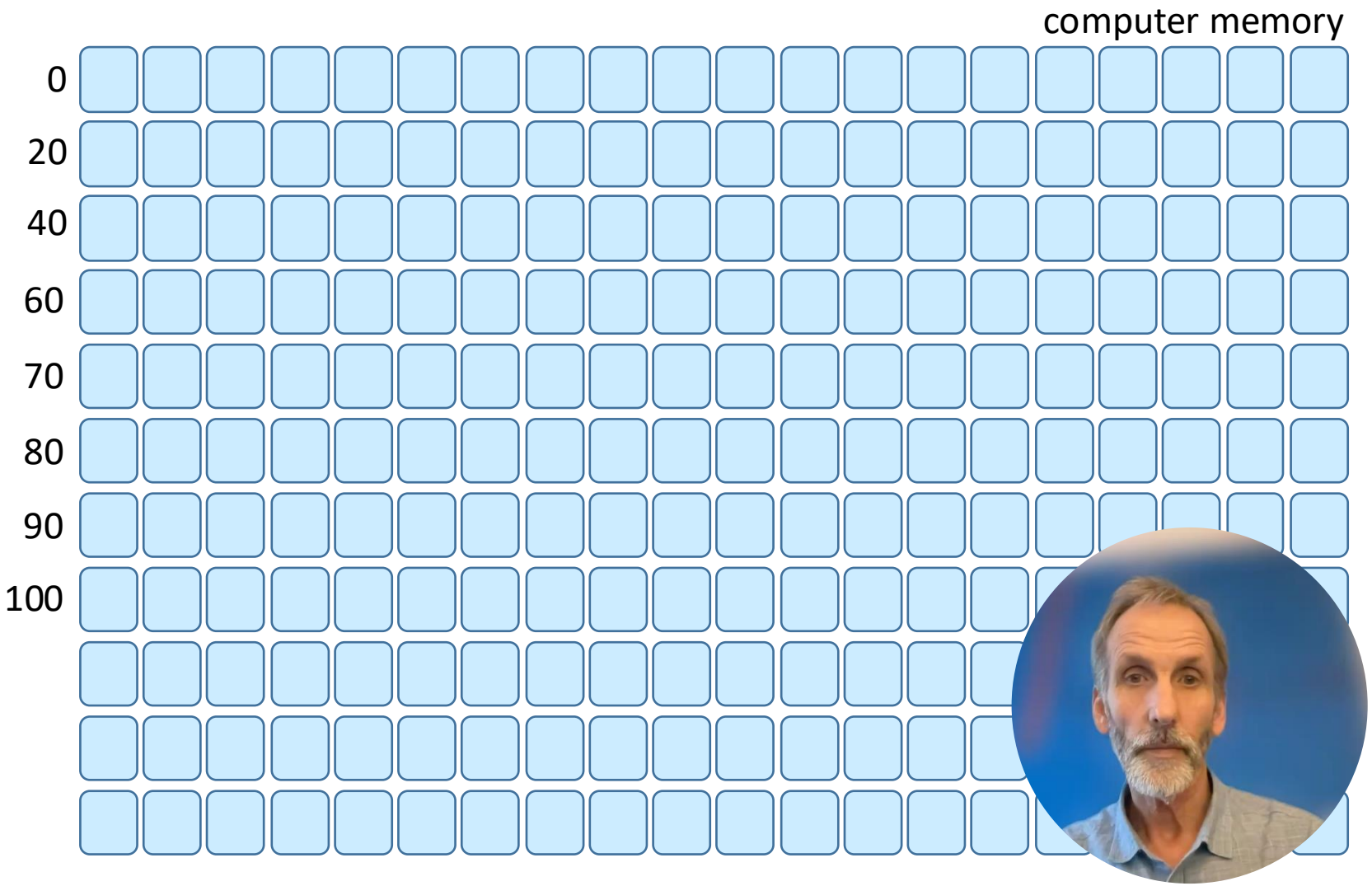
Pointers and memory

University of Exeter

- Memory addressing
- Arrays as pointers
- Dynamic memory allocation



Memory addresses



lookup table

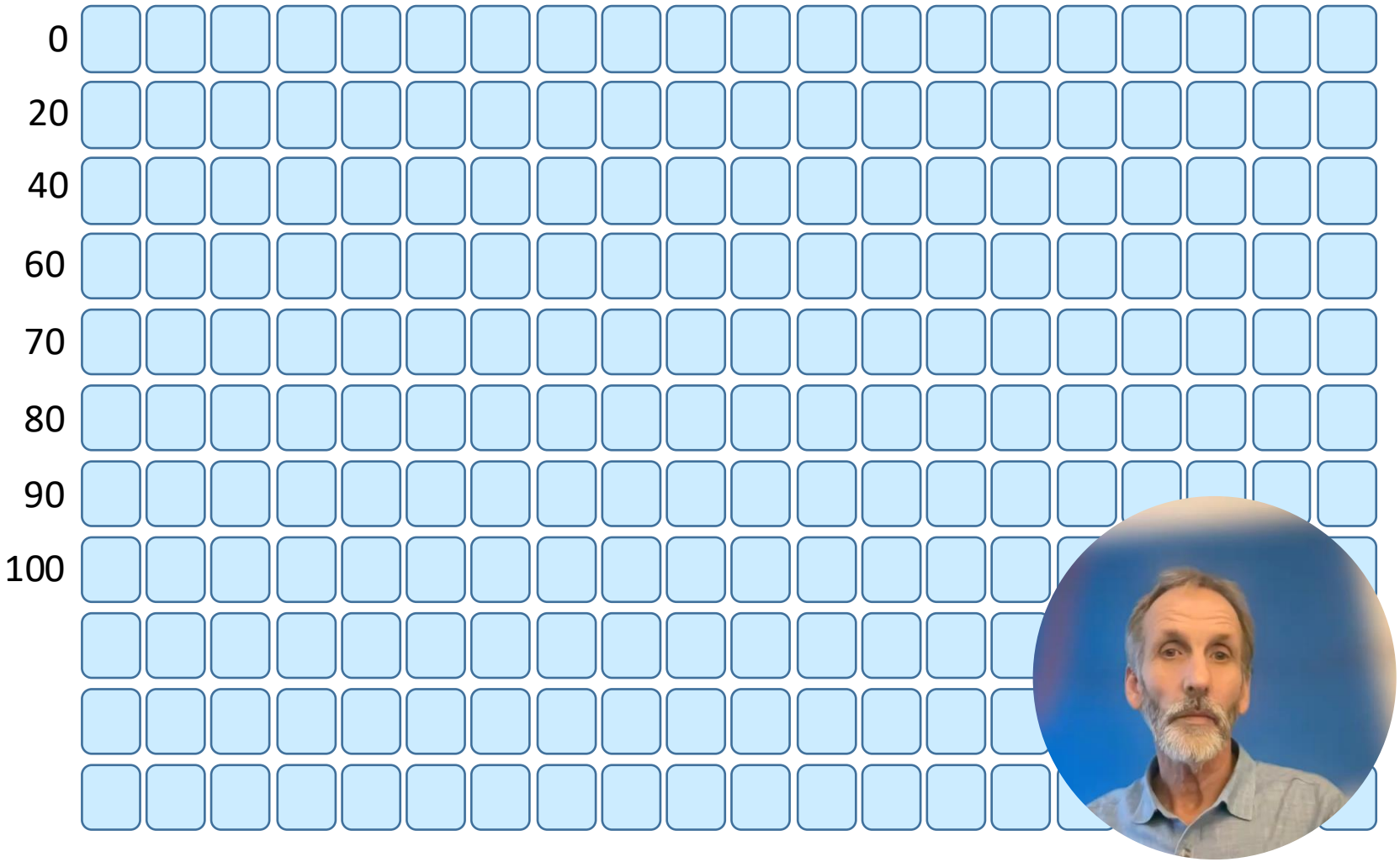
variable	type	address

Memory addresses

Declare a new variable:

```
int   x;
```

computer memory



lookup table

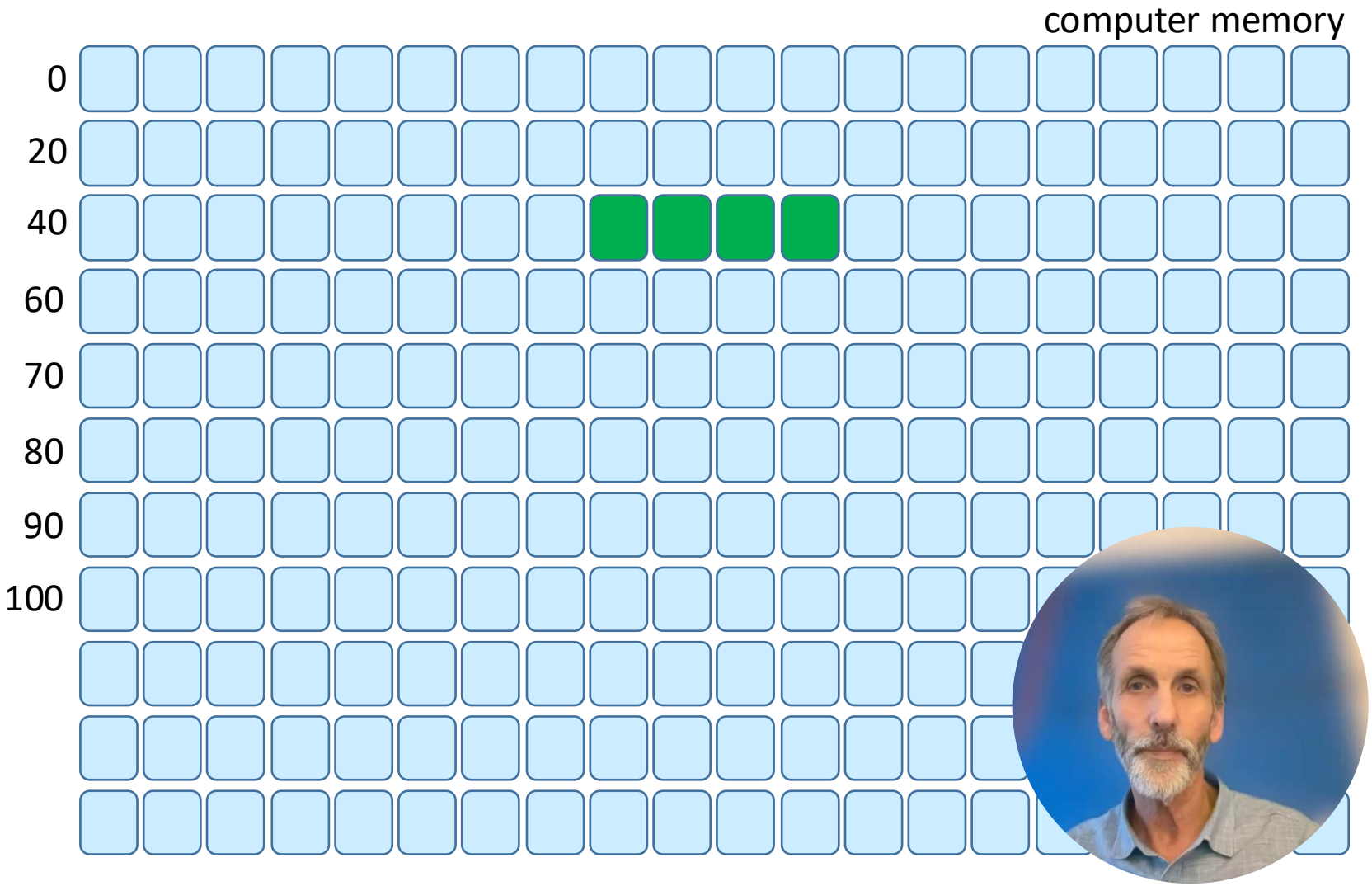
variable	type	address

Memory addresses

Declare a new variable:

```
int    x;
```

1. Allocate some memory to it
   `int` is 4 bytes in our system



lookup table

variable	type	address

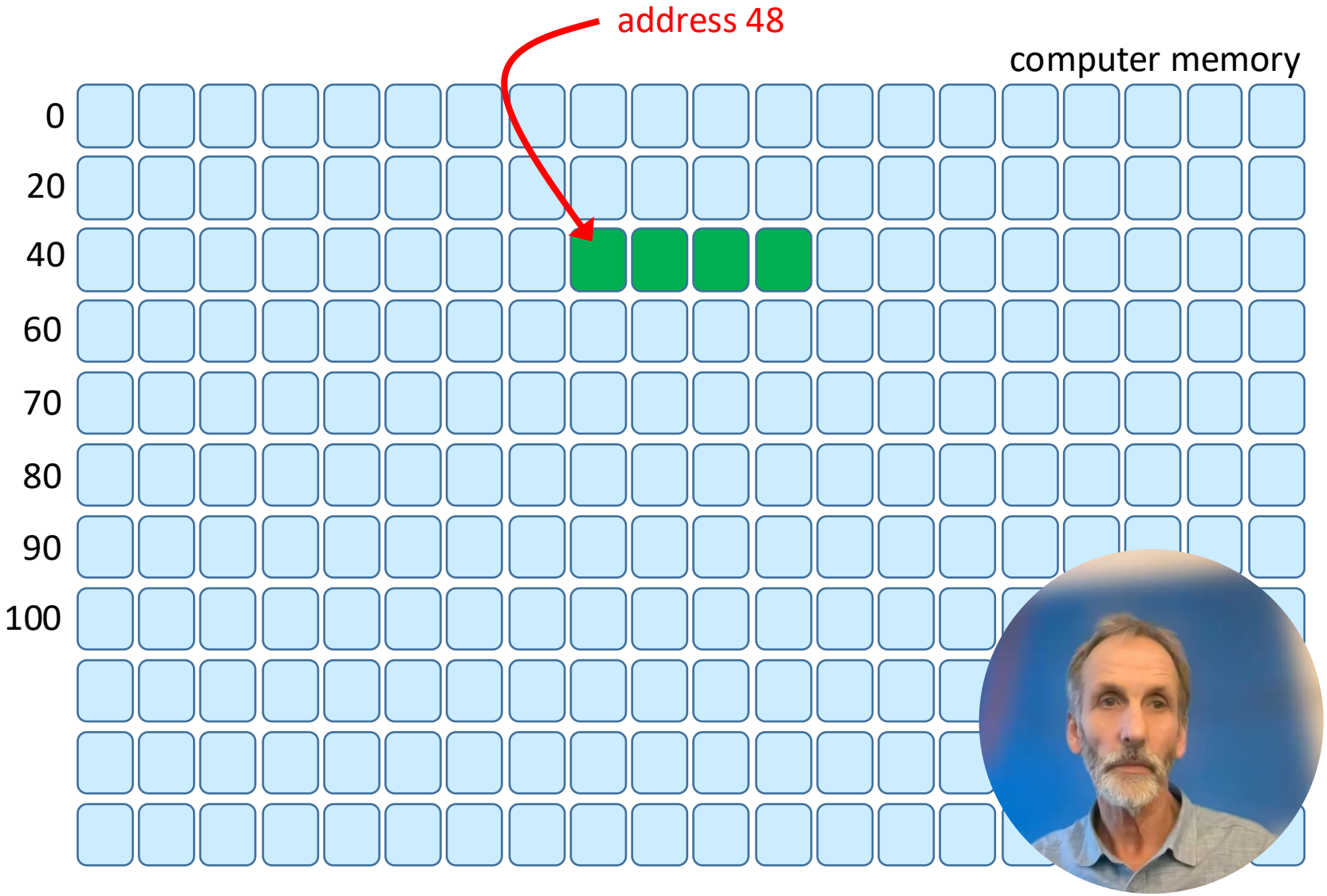
Memory addresses

Declare a new variable:

```
int    x;
```

1. Allocate some memory to it
   `int` is 4 bytes in our system

2. Add the info to the lookup table



lookup table

variable	type	address
x	int	48

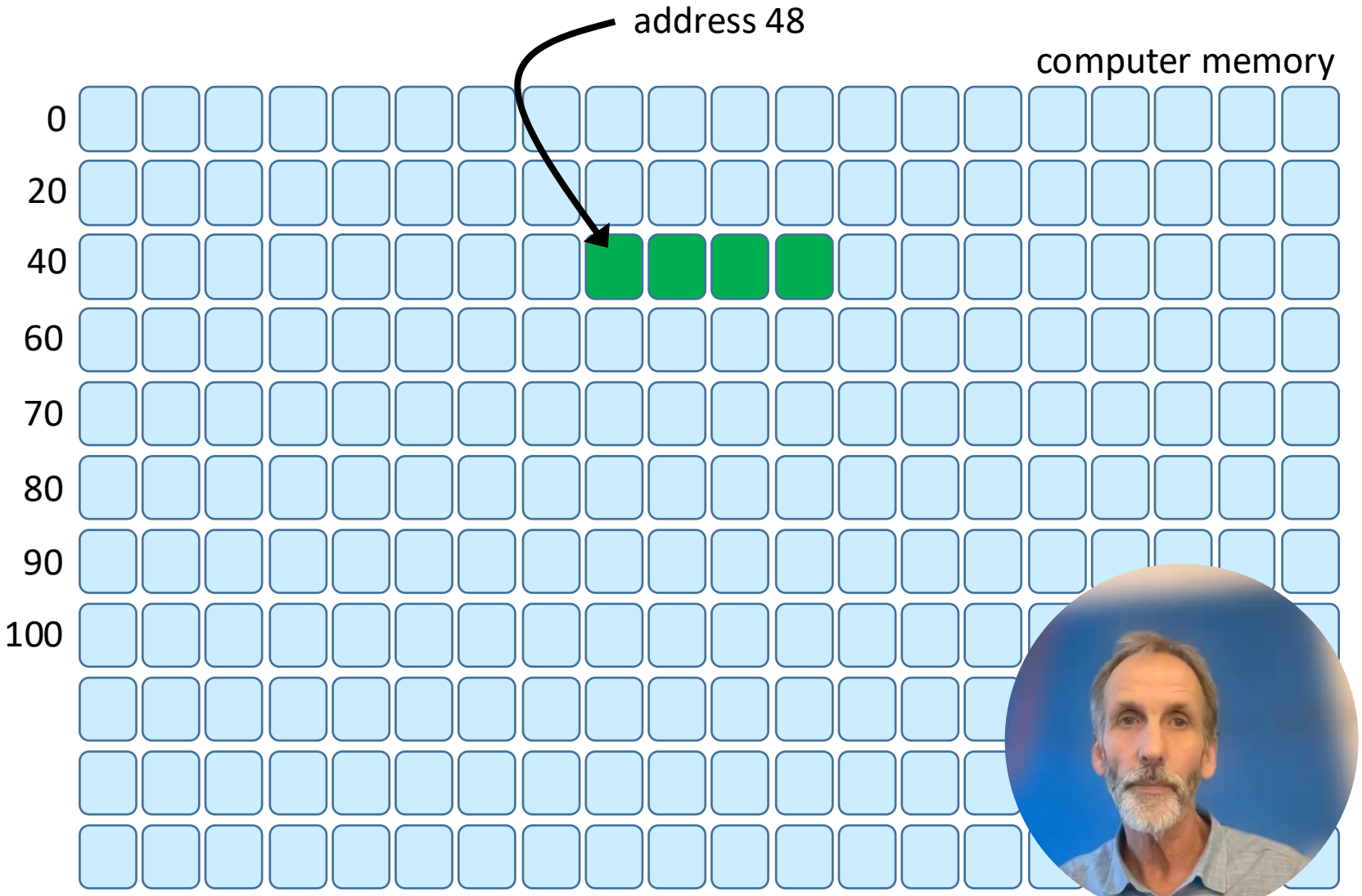
Memory addresses

Declare a new variable:

```
int   x;
```

Assign a value to the variable:

```
x = 2;
```



lookup table

variable	type	address
x	int	48

Memory addresses

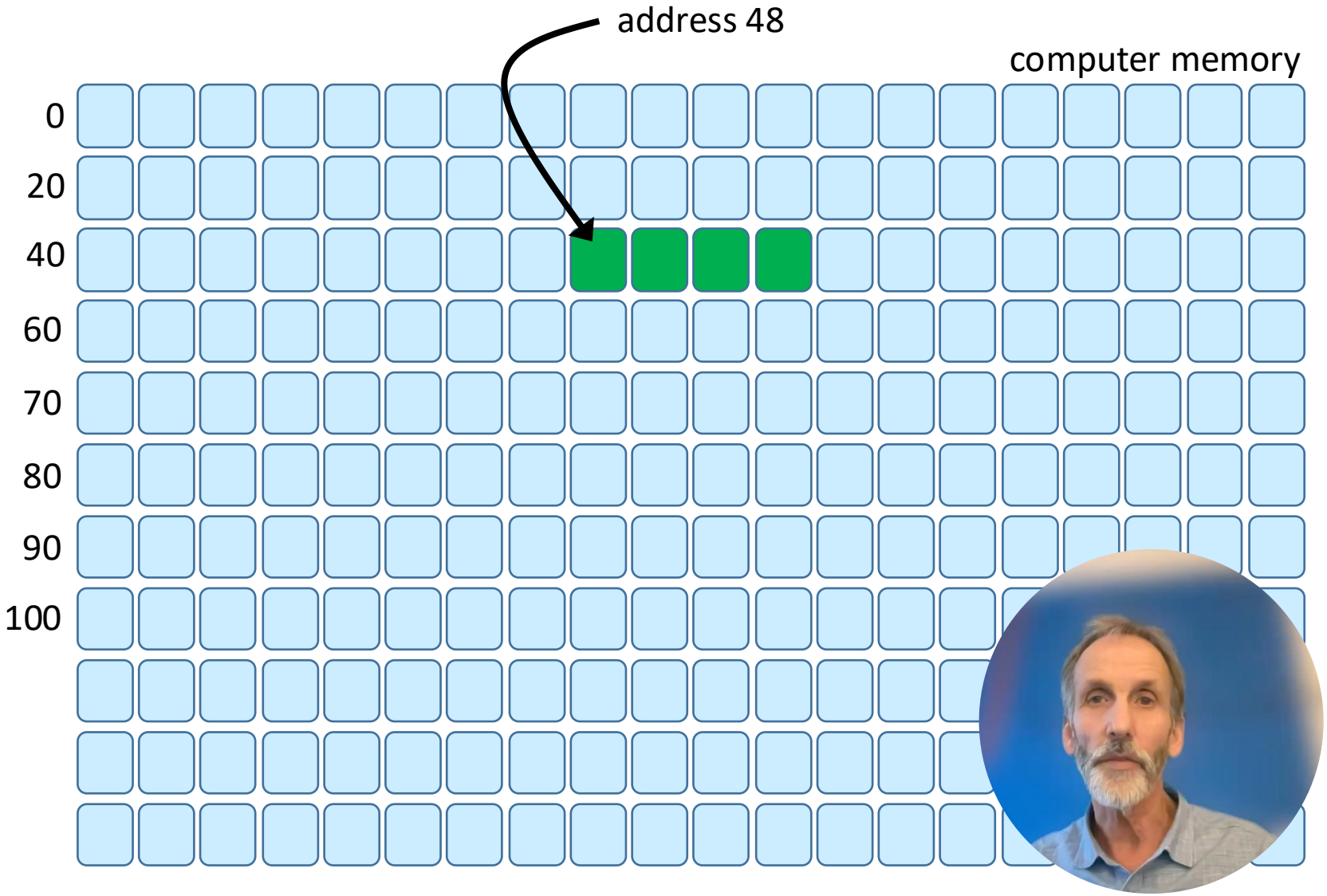
Declare a new variable:

```
int   x;
```

Assign a value to the variable:

```
x = 2;
```

1. Find **x** in the lookup table



lookup table

variable	type	address
x	int	48

Memory addresses

Declare a new variable:

```
int   x;
```

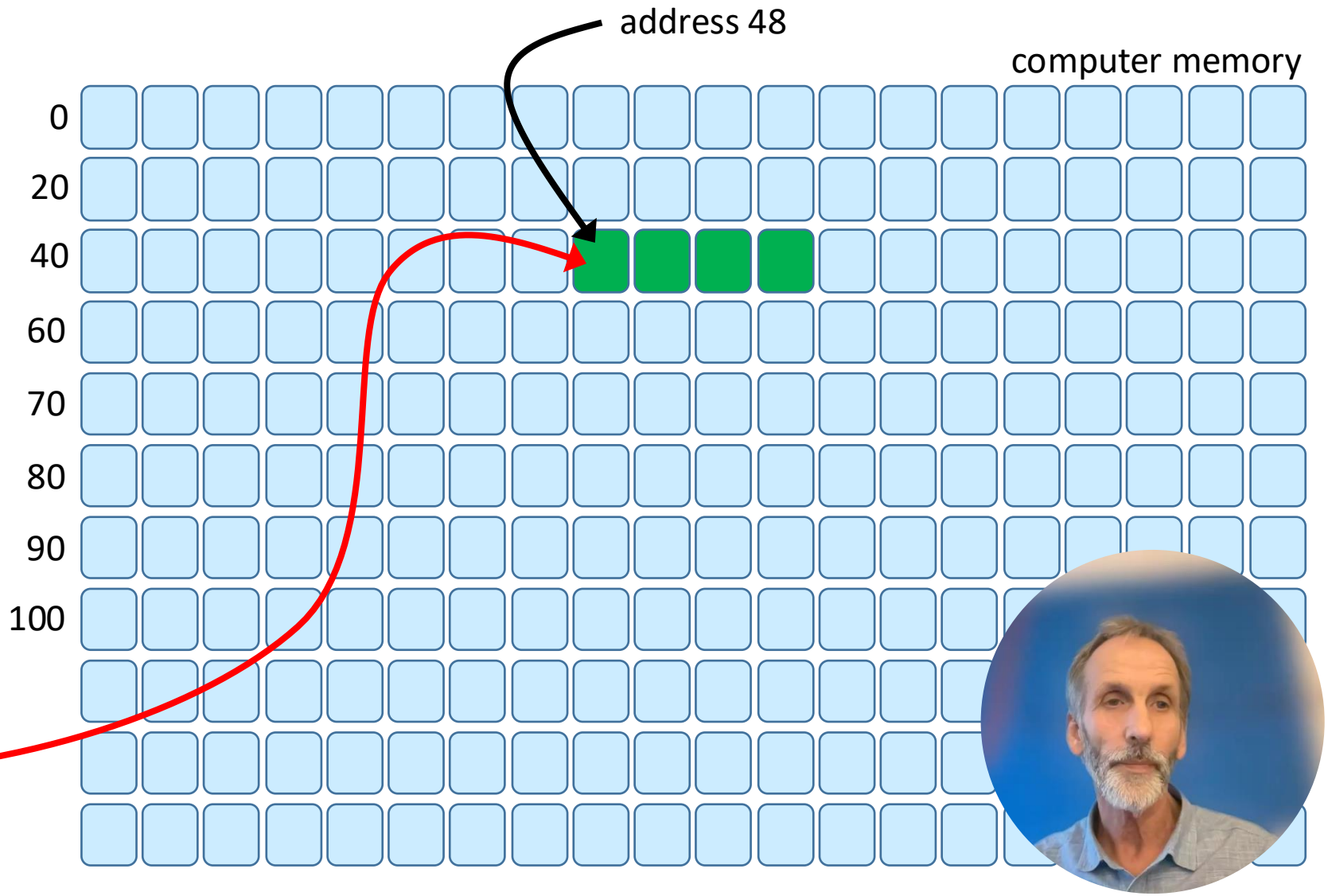
Assign a value to the variable:

```
x = 2;
```

1. Find **x** in the lookup table
2. Go to that memory address

lookup table

variable	type	address
x	int	48



Memory addresses

Declare a new variable:

```
int   x;
```

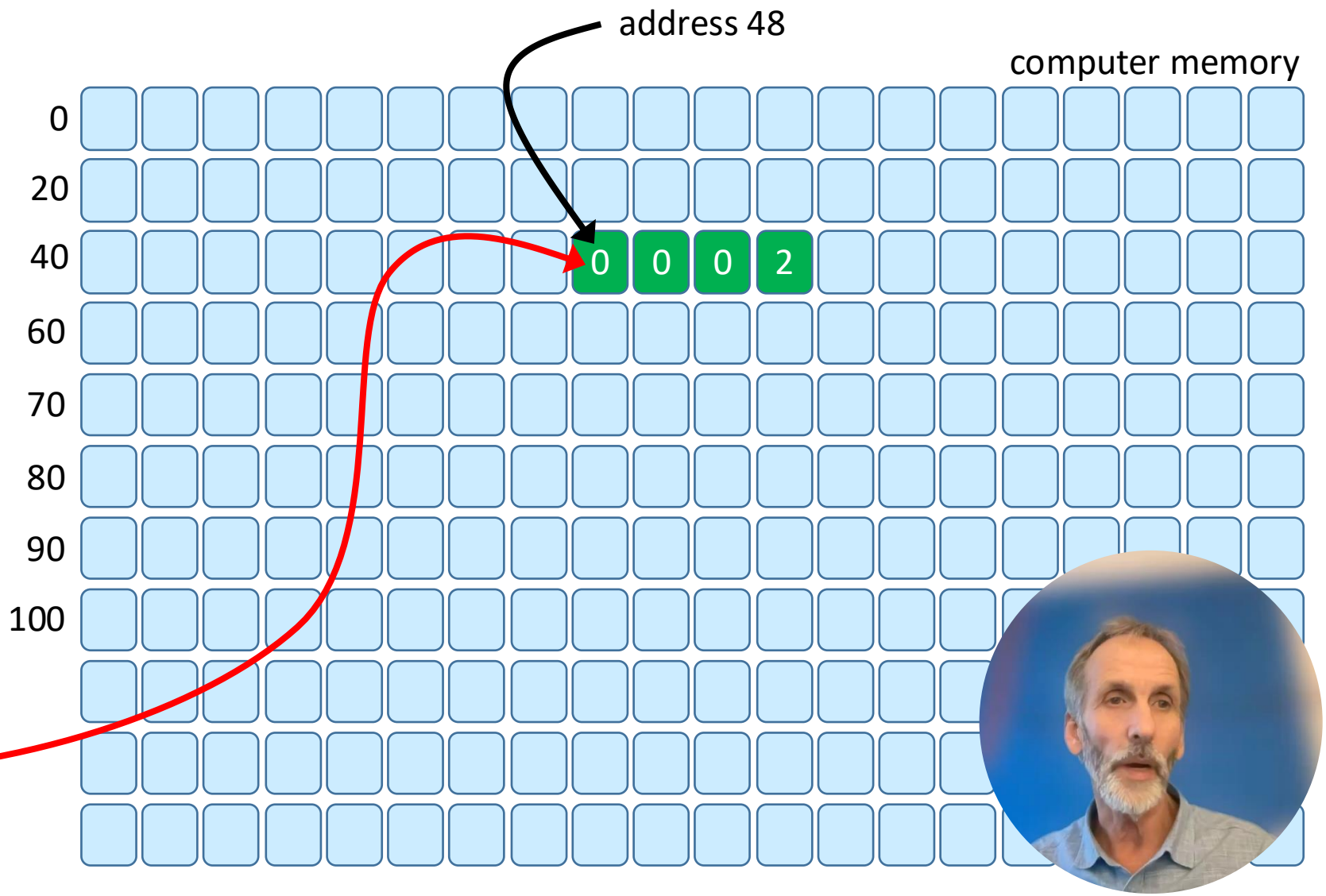
Assign a value to the variable:

```
x = 2;
```

1. Find **x** in the lookup table
2. Go to that memory address
3. Encode the assigned value

lookup table

variable	type	address
x	int	48



Pointers

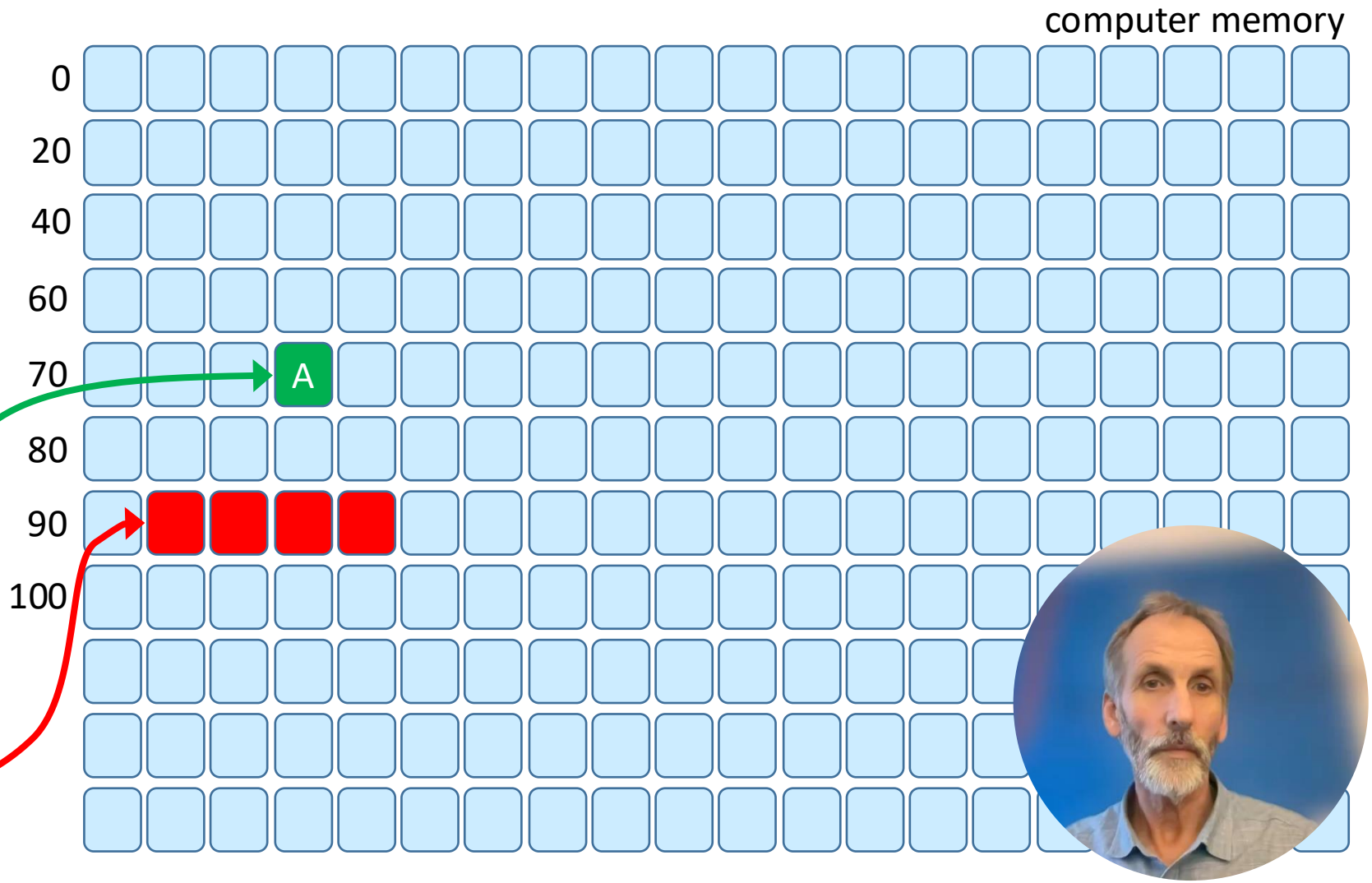
A pointer is a variable that holds the address of another variable in memory

`char  c = 'A';`	variable **c** is a character
`char *p;`	variable **p** is a pointer to a character
`char* p;`	emphasises that p is a ”pointer to char” datatype
`p = &c;`	**p** is a pointer to **ch**

Variable `p` contains the address where `c` is stored in memory.

Memory addresses

```
char   c = 'A';
char  *p;
```

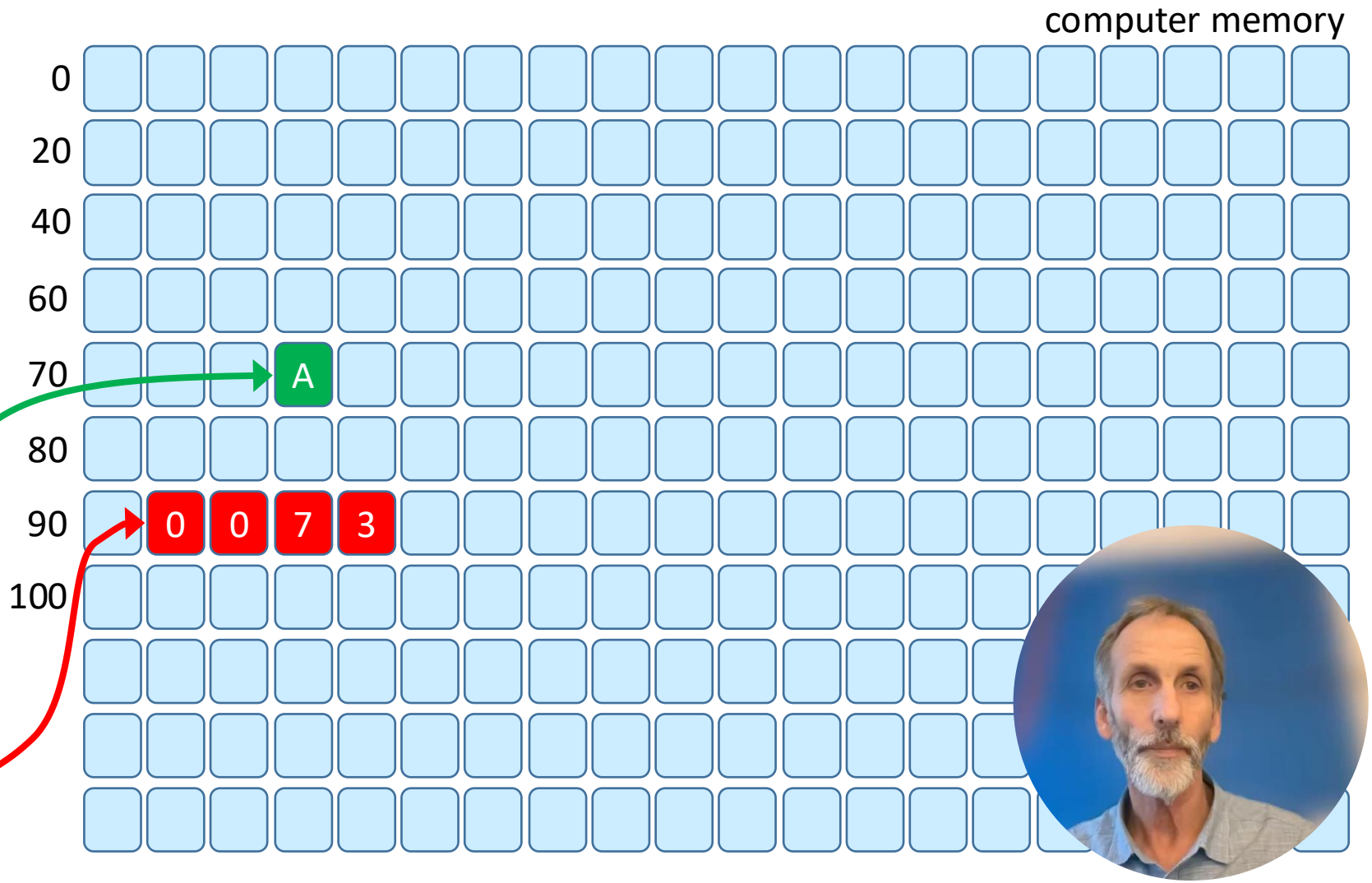


lookup table

variable	type	address
c	char	73
p	char *	91

Memory addresses

```
char   c = 'A';
char  *p;

p = &c;
```



lookup table

variable	type	address
c	char	73
p	char *	91

& and *

University of Exeter

- if `x` is any datatype, then

  `&x`
  is the location in memory (its “address”) where `x` is currently stored

- if `x` is a pointer, then

  `<datatype> *x;`
  declares `x` to contain the memory address of a variable of type <datatype>

- Dereferencing

  `*x`
  Means go to the memory address stored in `x` and bring back the value (of type
  stored there

Memory addresses

```
char   c = 'A';
char  *p;

p = &c;
```

p and **&c**

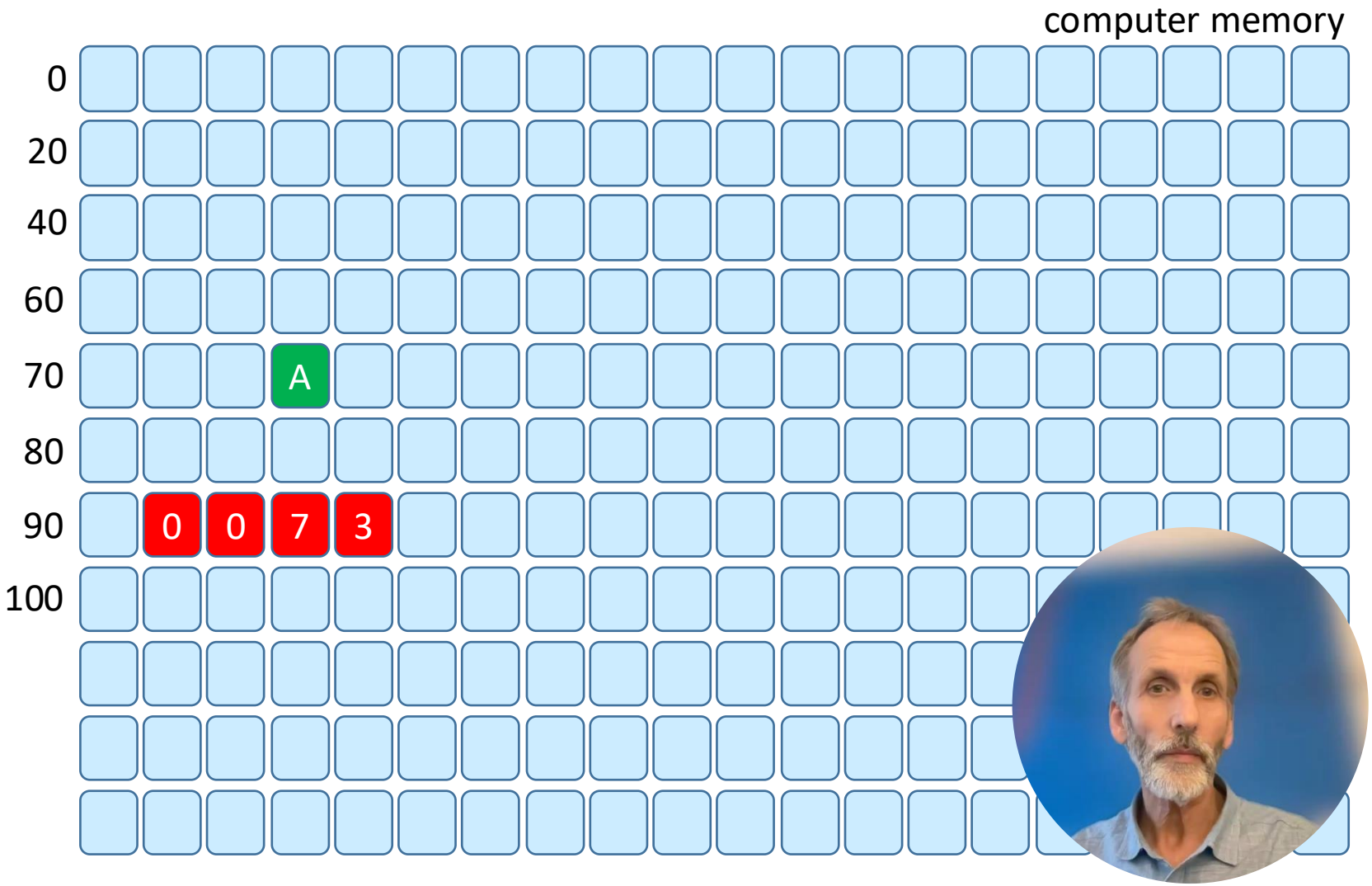
73

***p** and **c**

A

lookup table

variable	type	address
c	char	73
p	char *	91



Addresses and pointers

```
char c = 'A';
char d = 'D';
char *p;                /* Pointer to a char */
char *q;
char **x;               /* Pointer to a pointer to a char */
```

Addresses and pointers

University of Exeter

```
char c = 'A';
char d = 'D';
char *p;                /* Pointer to a char */
char *q;
char **x;               /* Pointer to a pointer to a char */

p = &c;                 /* p contains the address of c */

printf("p = %p\n", p);
printf("*p = %c\n", *p);
```

```
p = 0x1e224eff
*p = A
```

/* Addresses printed in hex */

Addresses and pointers

University of Exeter

```
char c = 'A';
char d = 'D';
char *p;                /* Pointer to a char */
char *q;
char **x;               /* Pointer to a pointer to a char */

p = &c;                /* p contains the address of c */

printf("p = %p\n", p);
printf("*p = %c\n", *p);

printf("\npoint p at d\n");
p = &d;
printf("p = %p\n", p);
printf("*p = %c\n", *p);
```

```
p = 0x1e224eff
*p = A

point p at d

p = 0x1e224efe
*p = D
```

/* Addresses printed in hex */

Addresses and pointers

University of Exeter

```
char c = 'A';
char d = 'D';
char *p;                /* Pointer to a char */
char *q;
char **x;               /* Pointer to a pointer to a char */

p = &c;                /* p contains the address of c */

printf("p = %p\n", p);
printf("*p = %c\n", *p);

printf("\npoint p at d\n");
p = &d;
printf("p = %p\n", p);
printf("*p = %c\n", *p);

printf("\n\nSet q = p\n");
q = p;
printf("q = %p\n", q);
printf("*q = %c\n", *q);
```

```
p = 0x1e224eff
*p = A

point p at d

p = 0x1e224efe
*p = D


Set q = p

q = 0x1e224efe
*q = D
```

/* Addresses printed in hex */

Addresses and pointers

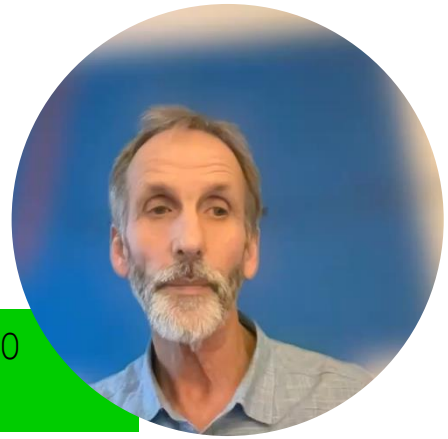
University of Exeter

```
char c = 'A';
char d = 'D';
char *p;                /* Pointer to a char */
char *q;
char **x;               /* Pointer to a pointer to a char */

p = &c;                /* p contains the address of c */

printf("p = %p\n", p);
printf("*p = %c\n", *p);

printf("\npoint p at d\n");
p = &d;
printf("p = %p\n", p);
printf("*p = %c\n", *p);

printf("\n\nSet q = p\n");
q = p;
printf("q = %p\n", q);
printf("*q = %c\n", *q);

x = &p;
printf("\n\nx = %p      &p = %p\n",x, &p);
printf("**x = %c\n", **x);
```

```
p = 0x1e224eff
*p = A

point p at d

p = 0x1e224efe
*p = D


Set q = p

q = 0x1e224efe
*q = D
```

/* Addresses printed in hex */

```
x = 0x1e224ef0      &p = 0x1e224ef0
**x = D
```



& and *

University of Exeter

- if `x` is any datatype, then

  `&x`
  is the location in memory (its “address”) where `x` is currently stored

- if `x` is a pointer, then

  `<datatype> *x;`
  declares `x` to contain the memory address of a variable of type <datatype>

- Dereferencing

  `*x`
  Means go to the memory address stored in `x` and bring back the value (of type stored there

Memory addresses

```
char   ch = 'A';
char  *p;

p = &ch;
```

p and **&ch**

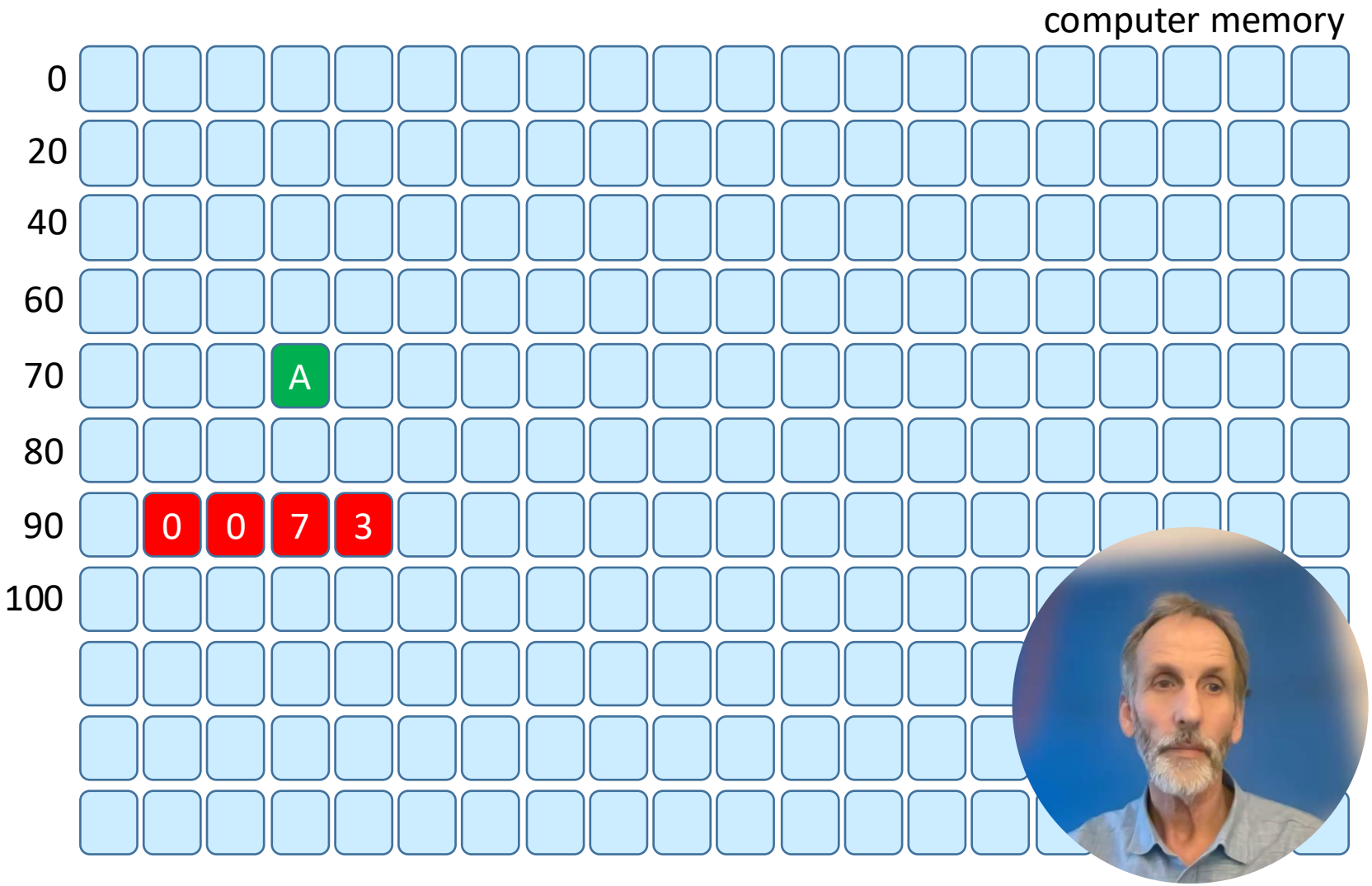
73

***p** and **ch**

A

lookup table

variable	type	address
ch	char	73
p	char *	91



sizeof a pointer

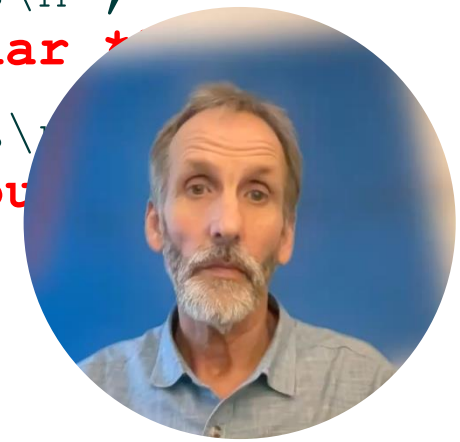
The amount of memory used to store a pointer depends on the computer, operating system, compiler setup, etc. Normally:

- Windows 32-bit
  32 bits = 4 bytes
- Windows 64-bit
  64 bits = 8 bytes

Our Linux servers:

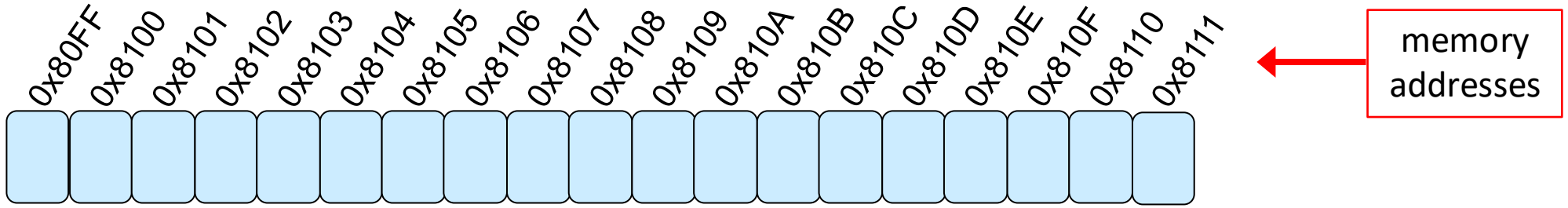
```
printf("size of a char   = %d; pointer to a char   = %d bytes\n",
                                  sizeof(char),   sizeof(char *
printf("size of a double = %d; pointer to a double = %d bytes\
                                  sizeof(double), sizeof(dou
```

```
size of a char   = 1; pointer to a char   = 8 bytes
size of a double = 8; pointer to a double = 8 bytes
```



Memory allocation example

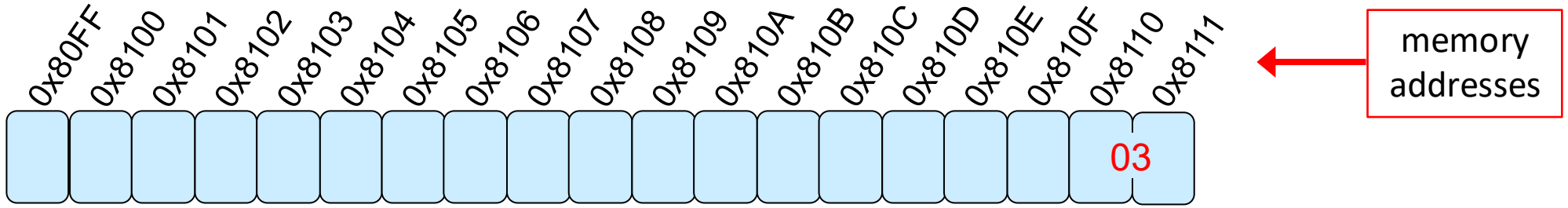
In this example assume that a pointer is 4 bytes



Memory allocation example

In this example assume that a pointer is 4 bytes

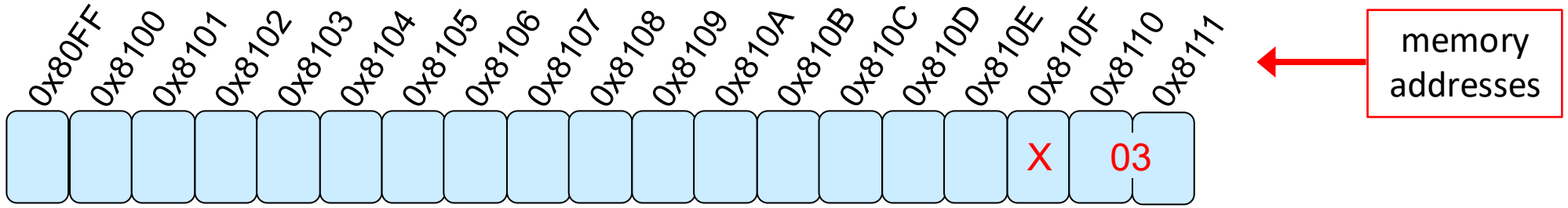
```
short int   a = 3;
```



Memory allocation example

In this example assume that a pointer is 4 bytes

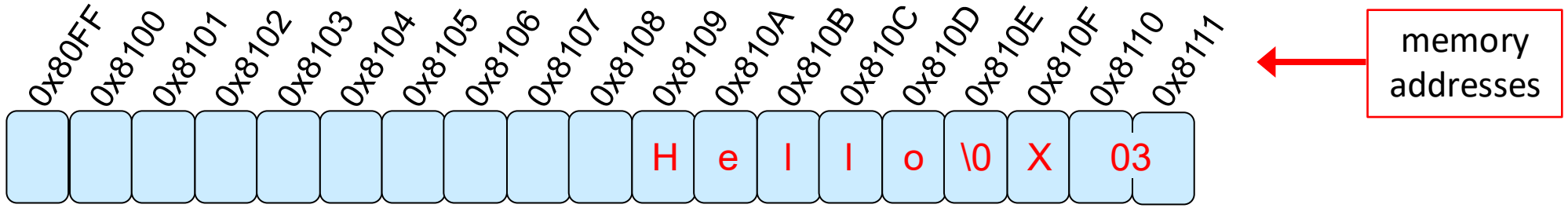
```
short int    a = 3;
char         b = 'X';
```



Memory allocation example

In this example assume that a pointer is 4 bytes

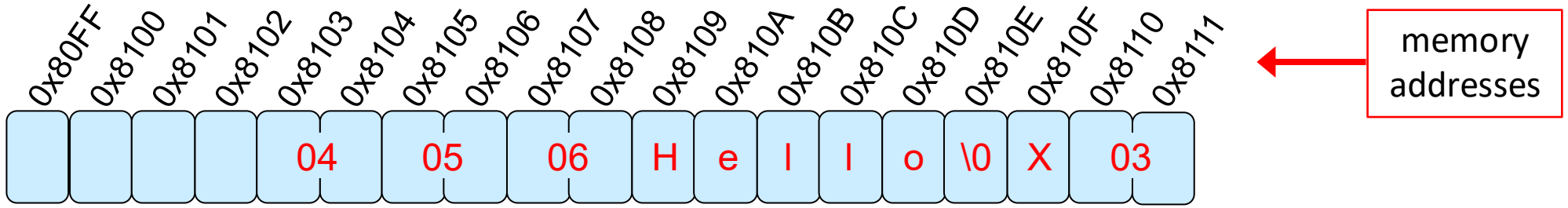
```
short int    a = 3;
char         b = 'X';
char         str[] = "Hello";
```



Memory allocation example

In this example assume that a pointer is 4 bytes

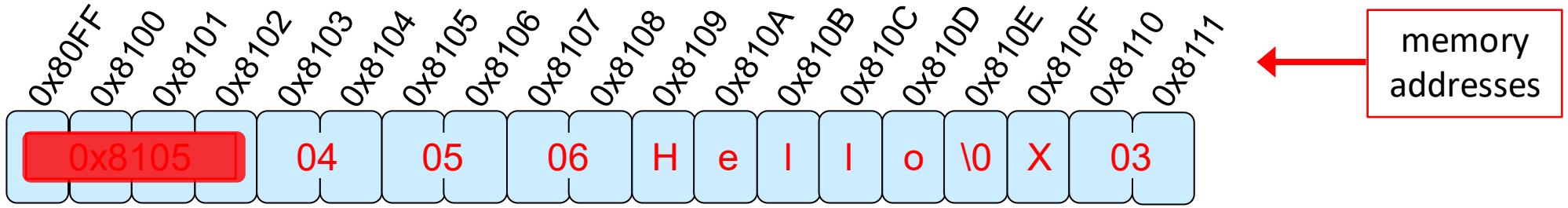
```
short int    a = 3;
char         b = 'X';
char         str[] = "Hello";
short int    c[3] = {4, 5, 6};
```



Memory allocation example

In this example assume that a pointer is 4 bytes

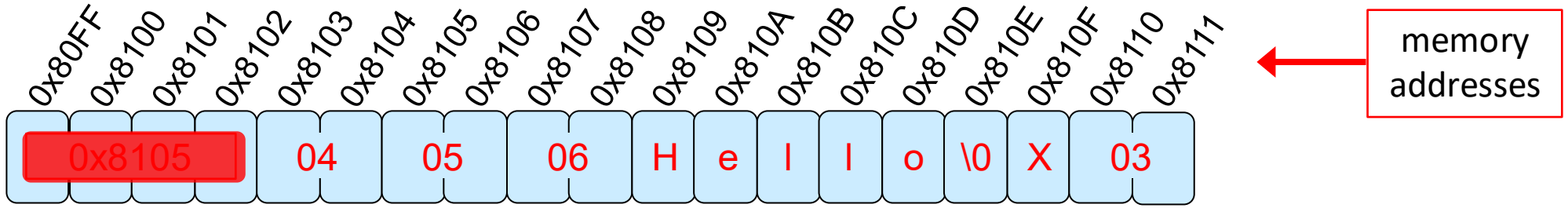
```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;
```



Memory allocation example

In this example assume that a pointer is 4 bytes

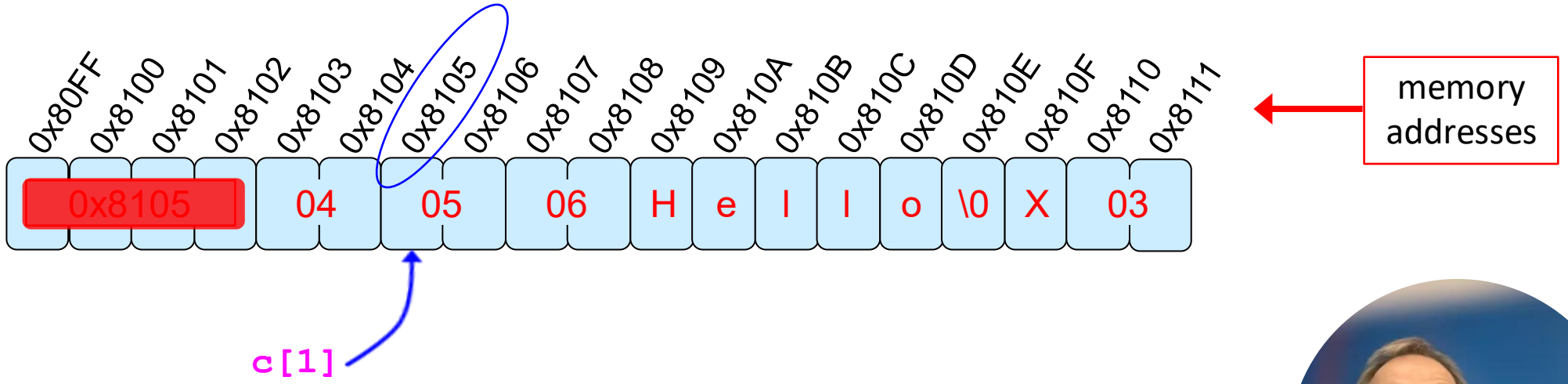
```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```



Memory allocation example

In this example assume that a pointer is 4 bytes

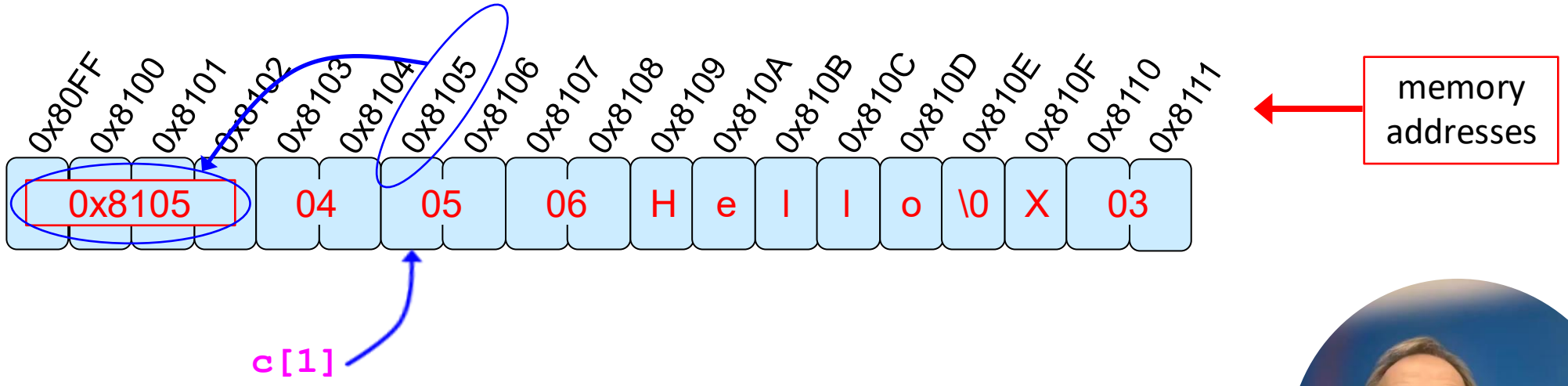
```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```



Memory allocation example

In this example assume that a pointer is 4 bytes

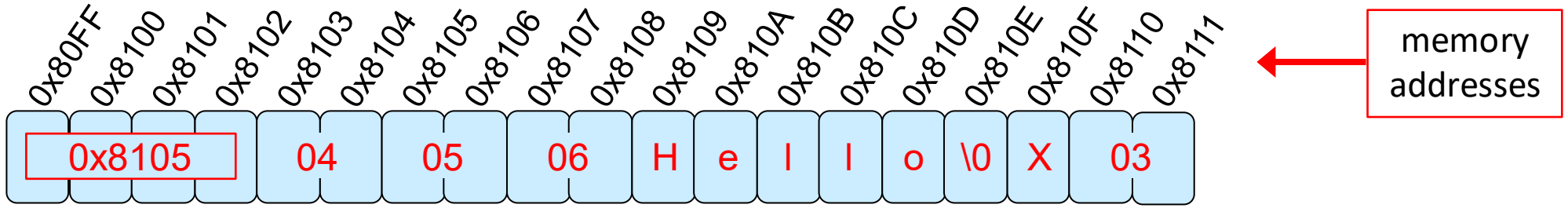
```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```



Memory allocation example

In this example assume that a pointer is 4 bytes

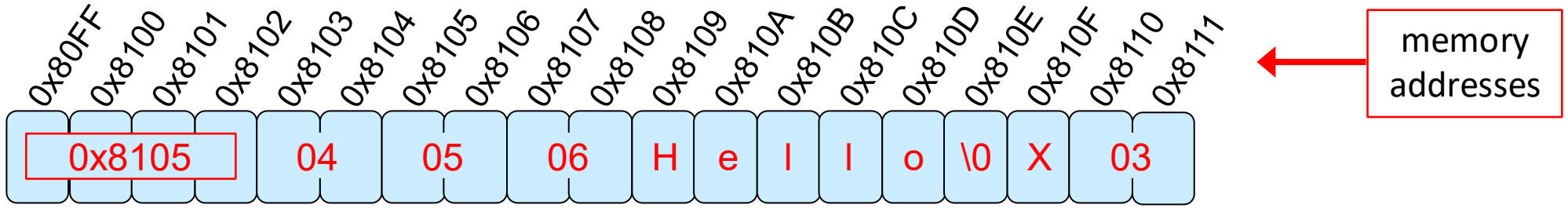
```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```



Memory allocation example

In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```

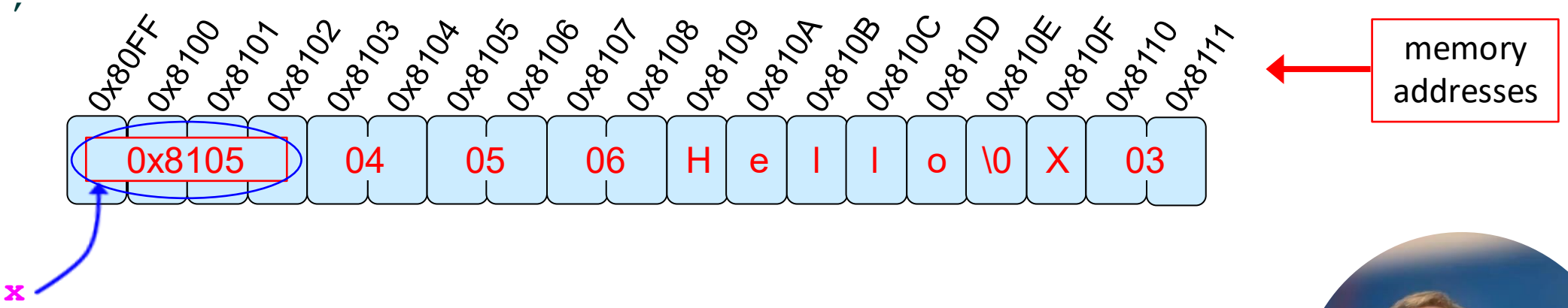



```
printf("x points to the value %d\n", *x);
```

Memory allocation example

In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```

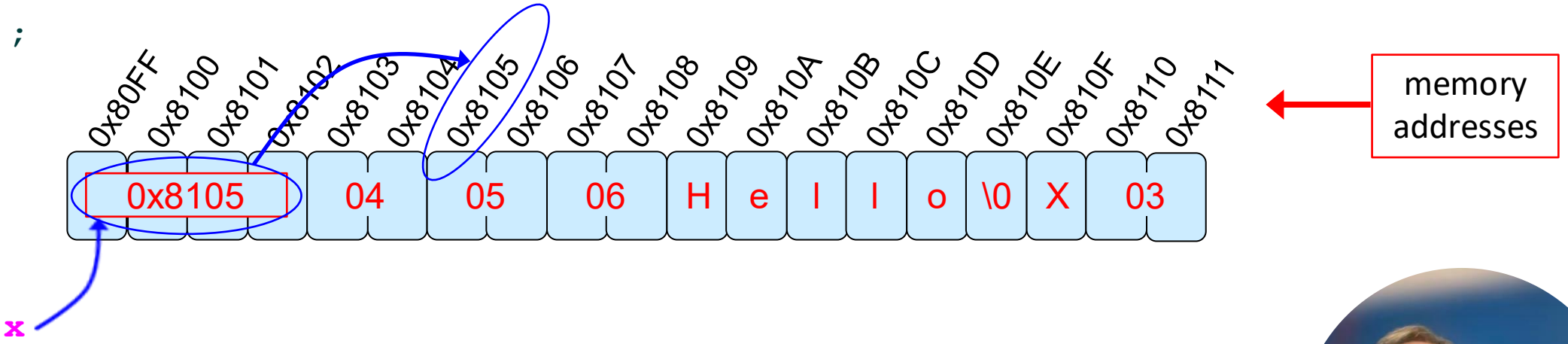



```
printf("x points to the value %d\n", *x);
```

Memory allocation example

In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```

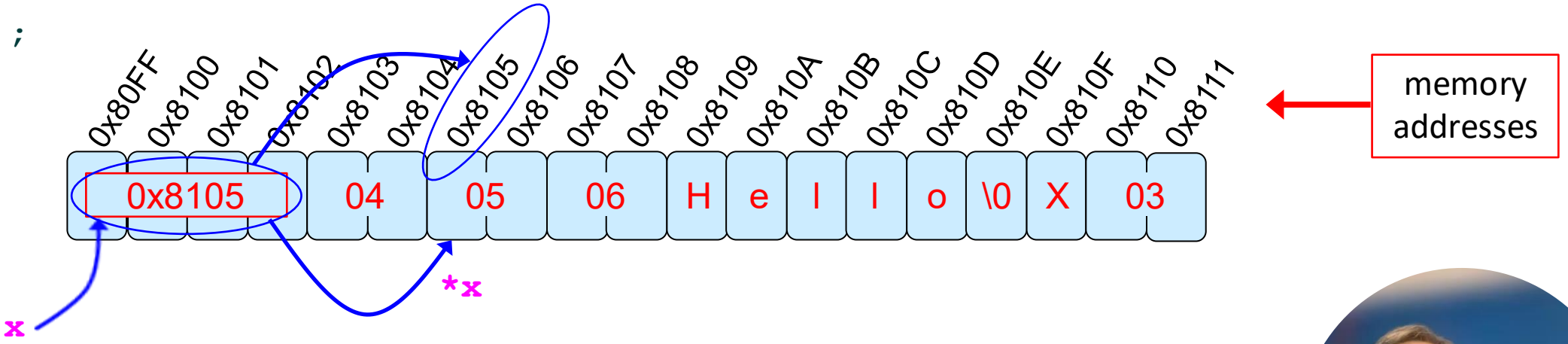



```
printf("x points to the value %d\n", *x);
```

Memory allocation example

In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```

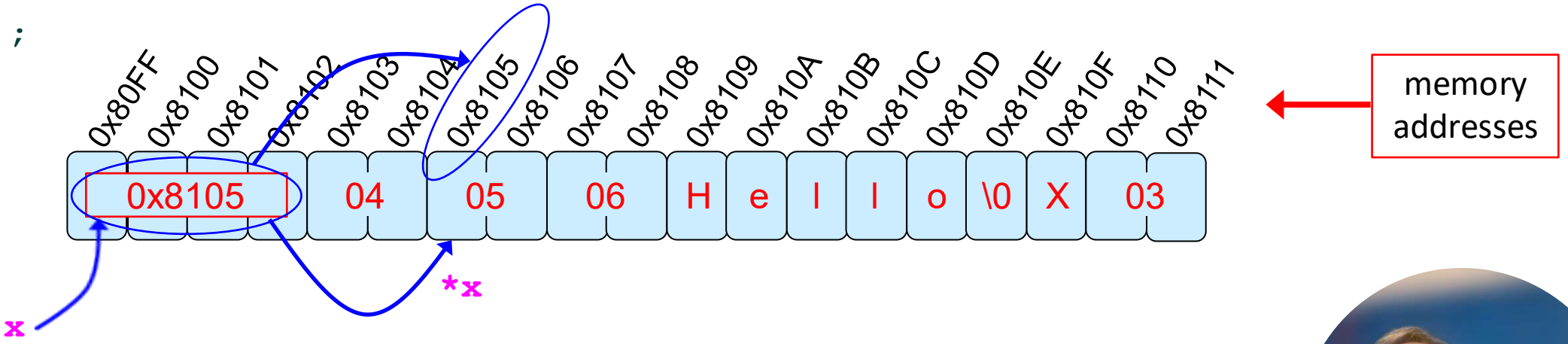



```
printf("x points to the value %d\n", *x);
```

Memory allocation example

In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```




```
printf("x points to the value %d\n", *x);
```

```
x points to the value 5
```

Summary

- C gives you access to the **memory address** of variable through **pointers**
- **&** gets the address of a variable
- *** dereferences** the pointer, giving the value of the variable pointed to

ECM2433: The C Family

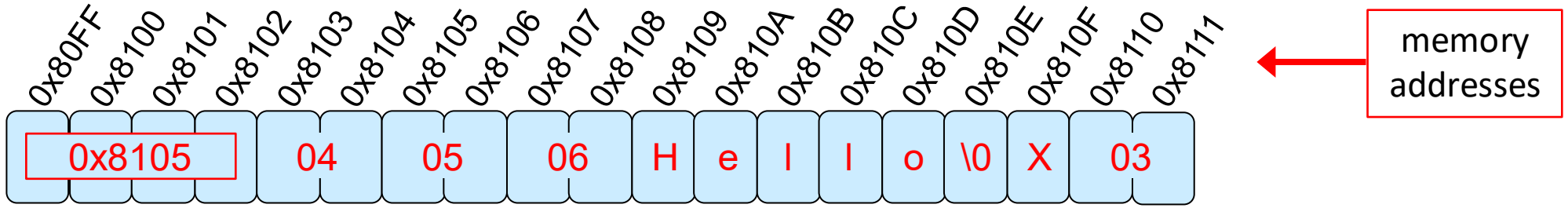
Pointer arithmetic

A memory address is just an integer, so we can perform arithmetic on it

Recall: memory allocation

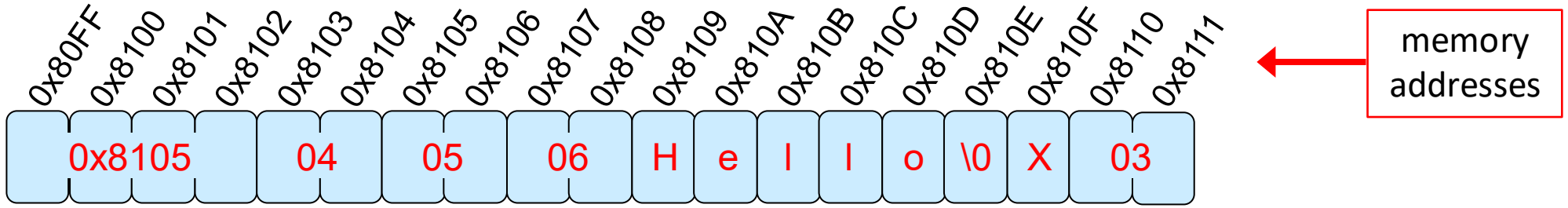
In this example assume that a pointer is 4 bytes

```
short int   a = 3;
char        b = 'X';
char        str[] = "Hello";
short int   c[3] = {4, 5, 6};
short int  *x;

x = &c[1];
```



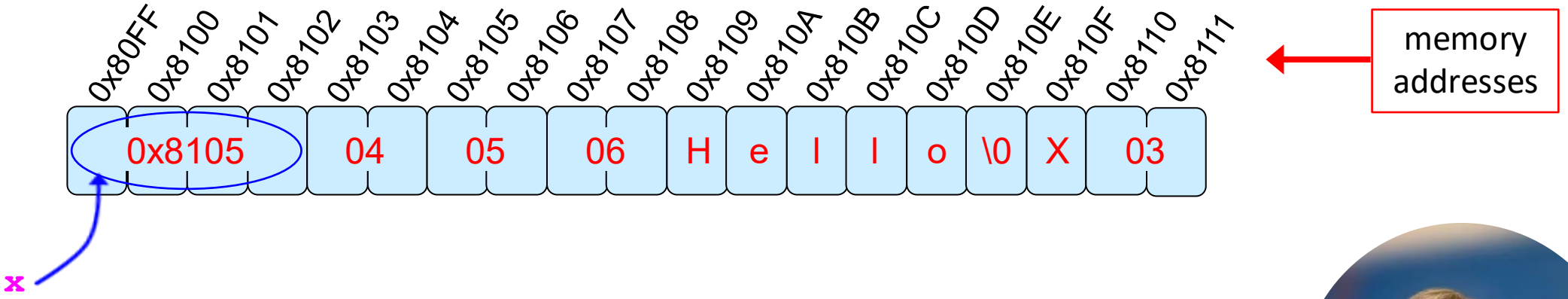
Pointer arithmetic

```
short int  *x;

x = &c[1];

x++;
```



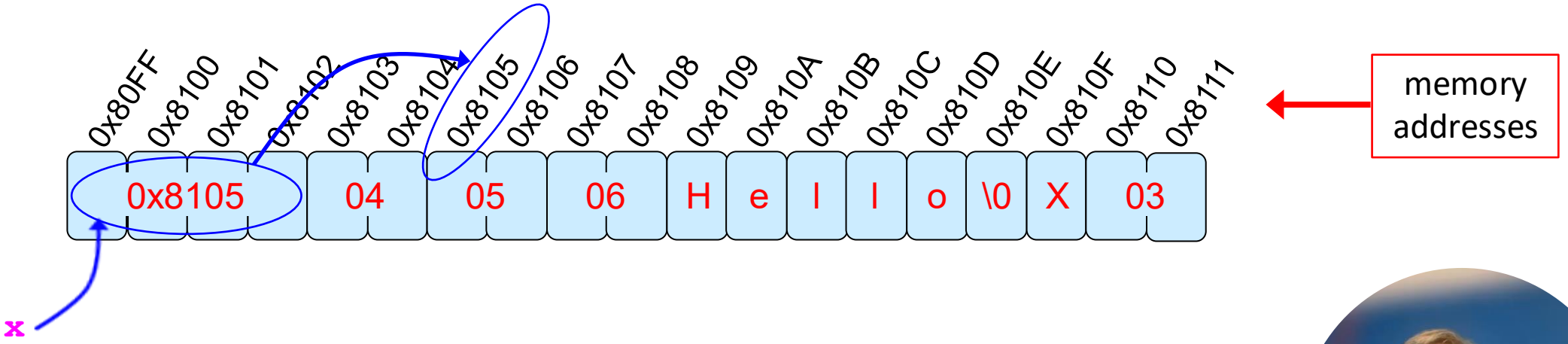
Pointer arithmetic

```
x++;
```



Pointer arithmetic

```
x++;
```



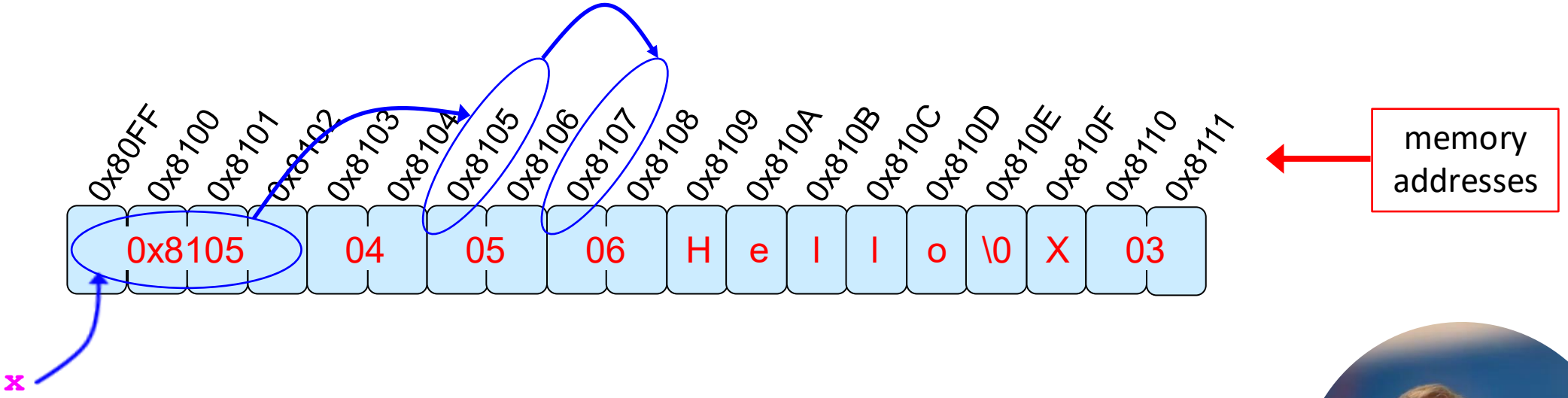
Pointer arithmetic

University of Exeter

`x++;`

Remember: short int *x
and short int is **2 bytes**

`x++` **means increment the address by 1 short int word; that is, "add 2 bytes"**



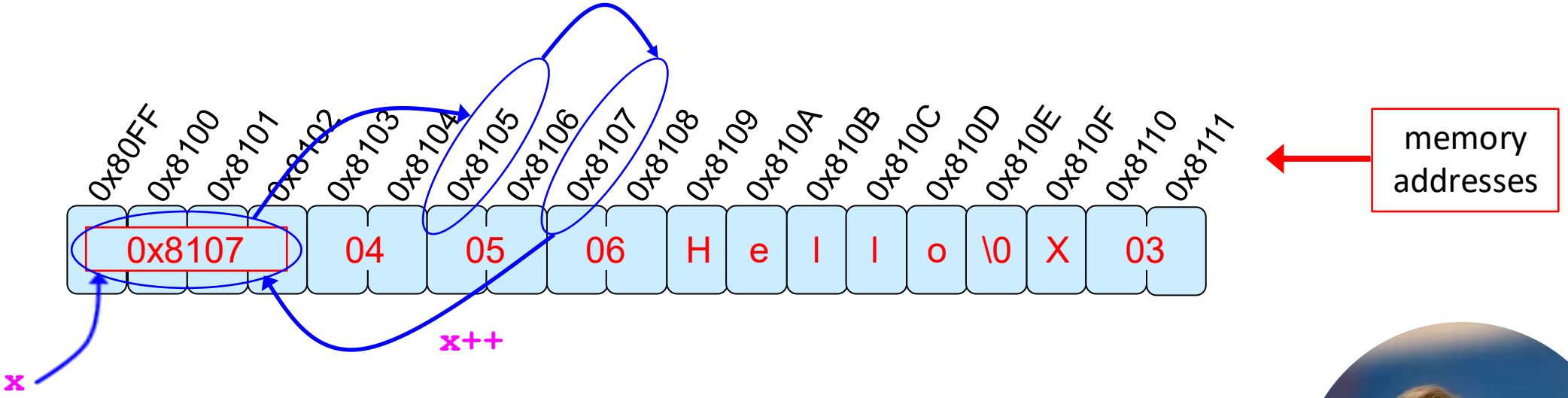
Pointer arithmetic

University of Exeter

`x++;`

Remember: short int *x
and short int is 2 bytes

`x++` means increment the address by 1 short int word; that is, "add 2 bytes"



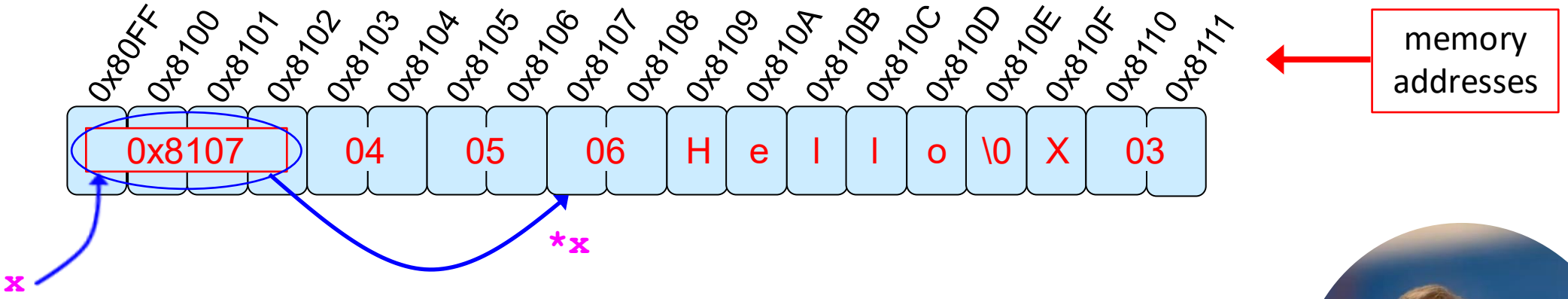
Pointer arithmetic

University of Exeter

`x++;`

Remember: short int *x
and short int is 2 bytes

`x++` means increment the address by 1 short int word; that is, "add 2 bytes"



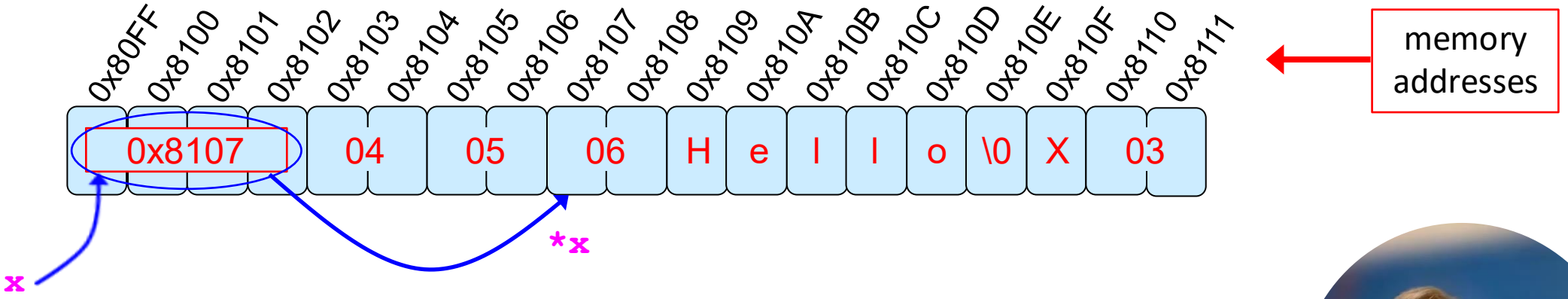
Pointer arithmetic

University of Exeter

`x++;`

Remember: short int *x
and short int is 2 bytes

`x++` means increment the address by 1 short int word; that is, "add 2 bytes"




```
printf("%d\n", *x);
```

6

Memory addresses

```
char   ch = 'A';
char  *p;

p = &ch;
```

p and **&ch**

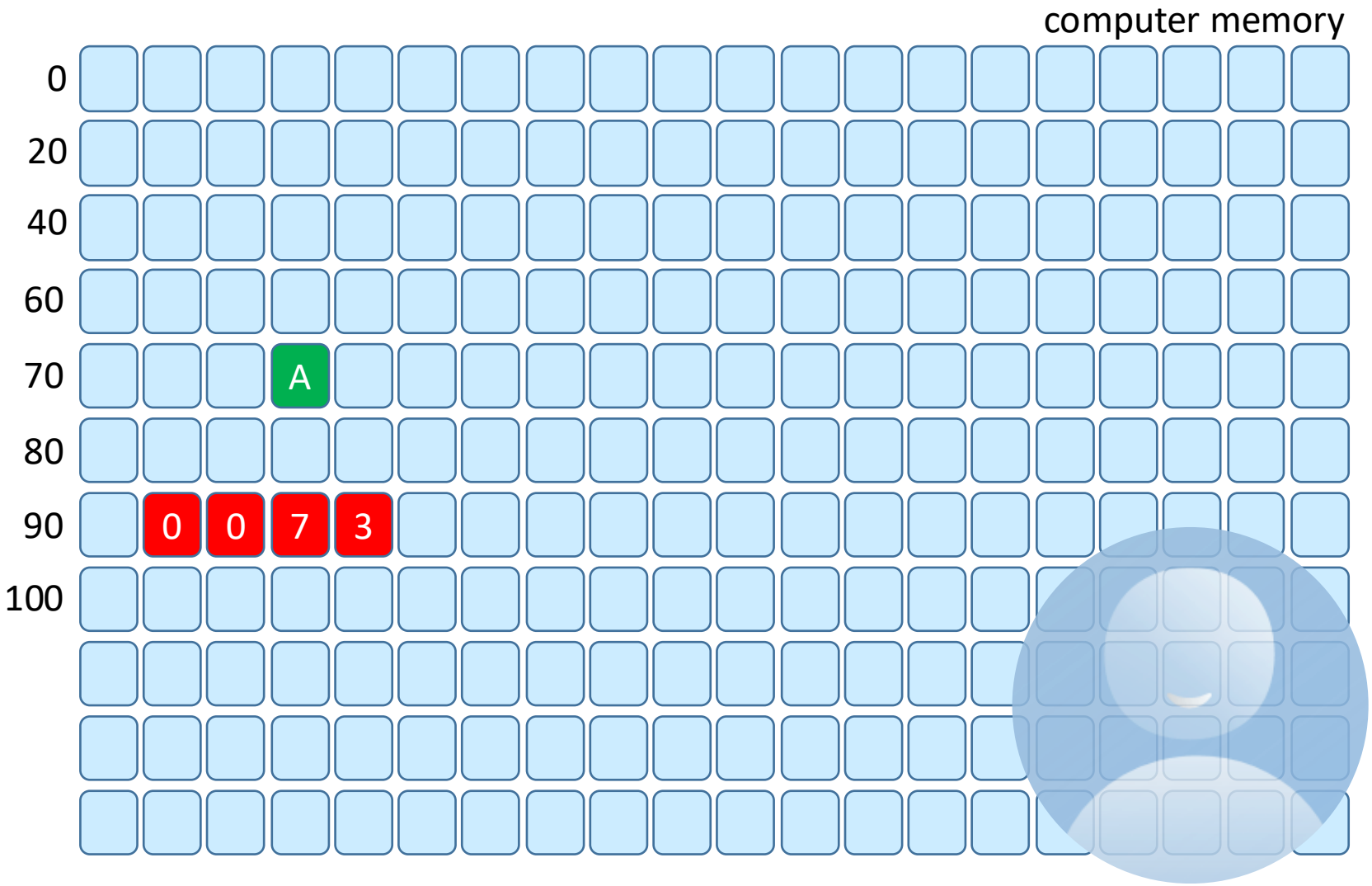
73

***p** and **ch**

A

lookup table

variable	type	address
ch	char	73
p	char *	91



Arrays (and strings): pointer equivalence

```
char  str[20] = "Hello world!";
char  *p = str;
```

Arrays (and strings): pointer equivalence

```
char  str[20] = "Hello world!";
char  *p = str;

printf("str is %s\n", str);
printf("p is %s\n", p);
```

```
str is Hello world!
p is Hello world!
```

Arrays (and strings): pointer equivalence

```
char  str[20] = "Hello world!";
char  *p = str;

printf("str is %s\n", str);
printf("p is %s\n", p);

p += 4;
printf("%s\n", p);
```

```
str is Hello world!
p is Hello world!
```

```
o world!
```

Arrays (and strings): pointer equivalence

```
char  str[20] = "Hello world!";
char  *p = str;

printf("str is %s\n", str);
printf("p is %s\n", p);
```

```
str is Hello world!
p is Hello world!
```

```
p += 4;
printf("%s\n", p);
```

```
o world!
```

```
*p = 'x';
printf("str is %s\n", str);
```

```
str is Hellx world!
```

Arrays (and strings): pointer equivalence

```
char  str[20] = "Hello world!";
char  *p = str;

printf("str is %s\n", str);
printf("p is %s\n", p);
```

```
str is Hello world!
p is Hello world!
```

```
p += 4;
printf("%s\n", p);
```

```
o world!
```

```
*p = 'x';
printf("str is %s\n", str);
```

```
str is Hellx world!
```

```
*p = '\0';
printf("str is %s\n", str);
```

```
str is Hell
```

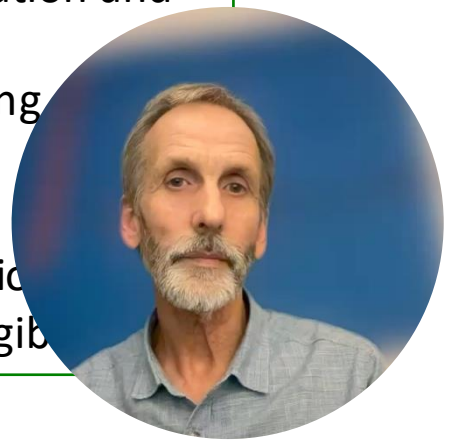
Loop idioms with pointers

University of Exeter

```
int i, N = 20;
int *p, powers[N];

/* Fill powers with squares */
for (i = 0; i < N; i++)
     powers[i] = i^2;

/* Add 2 to each element */
for (p = powers, i = 0; i < N; i++, p++)
    *p = *p + 2;

/* Add another 5 */
for (p = powers, i = 0; i < N; i++, p++)
    *p += 5;

/* Multiply each element by 10 */
for (p = powers, i = 0; i < N; i++)
    *p++ *= 10;
```

```
/* Make elements negative until an
 * element greater 1000 is encountered
 */
for (p = powers; *p < 1000; )
    *p++ *= -1;
```

- `powers[i] = foo;` involves finding the memory location of `powers`, adding `i` to it and going to that memory location and assigning `foo` to it.
- `*p++ = foo;` involves assigning whatever `p` currently points at and incrementing `p` ready for the next
- For large arrays, pointer manipulatio faster; the difference is often negligib



Arrays of strings

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char *string_array[] =   /* Array of pointers to char */
        { "Hello",
          "World",
          "Good",
          "Day" };

    int n = sizeof(string_array)/sizeof(char *);
    for (int i = 0; i < n; i++)
        printf("%d %lu %s\n", i, strlen(string_array[i]), string_array[i]);

    int r = 2;
    for (int c = 0; c < strlen(string_array[r]); c++)
        printf("%c", string_array[r][c]);
    printf("\n");

    char **s = string_array;
    for (int i = 0; i < n; i++, s++)
        printf("%d %lu %s\n", i, strlen(*s), *s);

    return 0;
}
```

```
0 5 Hello
1 5 World
2 4 Good
3 3 Day
Good
0 5 Hello
1 5 World
2 4 Good
3 3 Day
```

Runtime errors using pointers

University of Exeter

- ☞ Memory fault
- ☞ Segmentation fault

Caused by

- trying to access memory to which it is not allowed access
- trying to access memory in a way that is not allowed
  e.g. trying to overwrite the operating system

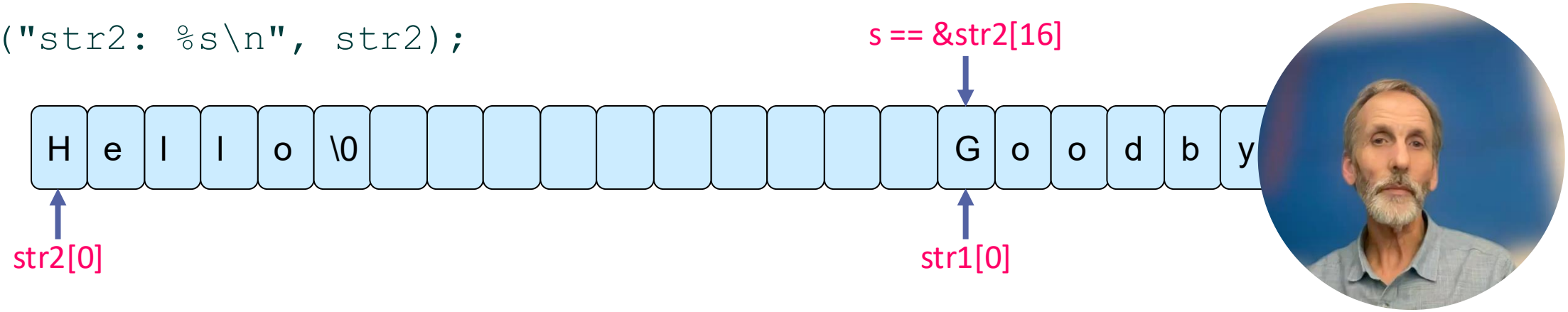
Runtime errors using pointers

University of Exeter

You might not get an error, you might just corrupt your own program.

```
char   str1[20] = "Goodbye";
char   str2[6]  = "Hello";
char  *s = str2;
s += 16;
*s = 'x';

printf("str1: %s\n", str1);
printf("str2: %s\n", str2);
```

```
str1: xoodbye
str2: Hello
```



University of Exeter

ECM2433: The C Family

Dynamic memory allocation

Void pointers

Void pointers can point to any datatype:

```
char   c = 'A';
float  n = 10.0;
void  *p;
```

p can point to anything...

```
p = &c;
printf("p points to %c\n", *(char *)p);
```

... but needs a cast when used

```
p = &n;
printf("p points to %f\n", *(float *)p);
```

Dynamic memory allocation

Sometimes you do not know at compile time how much memory you will need at runtime. Possible solutions:

1. Declare an array whose size is decided at runtime.

```
#include <string.h>

int main ()
{
    unsigned int n = some_calculation();
    char ch[n];
    strcpy(ch, "Elephant");
    return 0;
}
```

Dynamic memory allocation

Sometimes you do not know at compile time how much memory you will need at runtime. Possible solutions:

1. Declare an array whose size is decided at runtime.

```
#include <string.h>
#include <limits.h>

int main ()
{
    unsigned int n = UINT_MAX;
    char ch[n];
    strcpy(ch, "Elephant");
    return 0;
}
```

Dynamic memory allocation

Sometimes you do not know at compile time how much memory you will need at runtime. Possible solutions:

1. Declare an array whose size is decided at runtime.

```
#include <string.h>
#include <limits.h>

int main ()
{
    unsigned int n = UINT_MAX;
    char ch[n];
    strcpy(ch, "Elephant");
    return 0;
}
```

Memory fault(coredump)

Probably because there is not enough free, contiguous memory to allocate to the array.

Dynamic memory allocation

Sometimes you do not know at compile time how much memory you will need at runtime. Possible solutions:

1. Declare an array whose size is decided at runtime.
   But there is no way of knowing how much free contiguous memory is available.
   And you might not know how much is required before declaring the array.
2. Process the data in smaller chunks.
   Possible, but requires a lot of extra programming.
3. Dynamically allocate memory at runtime using pointers.
   Here we can check whether memory allocation was successful.

Dynamic memory allocation

Memory, from the heap, can be allocated to a variable at runtime:

```
#include <stdlib.h>
```

/* Prototype for malloc: `void *malloc(size_t size);` */

```
int *p = NULL;

p = (int *)malloc(23);
```

23 bytes

```
p = (int *)malloc(23*sizeof(int));
```

23 integers

Note the cast: the return datatype for malloc is a void pointer

`int *p = NULL;`
allocates 8 bytes (for a 64-bit system) to `p`

`p = (int *)malloc(23);`
finds 23 free bytes in memory, reserves it ,

and sets `p` to the memory address of the first of those bytes

Dynamic memory allocation

University of Exeter

There might not be enough memory:

```
p = (int *)malloc(100000000000000*sizeof(int));  not enough memory
printf("p = %p\n", p);
```

```
p = (nil)
```

You must **ALWAYS** trap the fact that memory was not allocated:

```
if ( !(p = (int *)malloc(100000000000000*sizeof(int))) )  {
   printf("Out of memory\n");
   exit(1);
}
```

Dynamic memory allocation

There might not be enough memory:

```
p = (int *)malloc(100000000000000*sizeof(int));  not enough memory
printf("p = %p\n", p);
```

```
p = (nil)
```

You must **ALWAYS** trap the fact that memory was not allocated:

```
if ( !(p = (int *)malloc(100000000000000*sizeof(int))) )  {
   printf("Out of memory\n");
   exit(1);
}
```

"always" = negative marks if you fail to trap this in coursework !

Variables and scope

Automatric variables

```
void myFunction ()
{
    int p = 1000;
}
```

☞ p goes out of scope at the end of the function and its allocated memory on the stack is released

Variables and scope

Dynamically allocated variables:

```
void myFunction()
{
    int *p = (int *)malloc(1000000);
}
```

☞ **Memory leak**

allocated memory is **NOT** released, even though p goes out of scope.

The 8 bytes (on a 64-bit system) occupied by `p` are released,

but the 1000000 bytes reserved for `p`, and to which `p` points, are not.

Free your memory

Whenever you use malloc, you must **ALWAYS** explicitly free the memory before the variable goes out of scope:

"always" means large penalties if you fail to do this in coursework!

```
void myProcedure()
{
    int *p = (int *)malloc(1000000);
    /* Use the memory for something */
    free(p);
}
```

Freeing memory returns it to the system, allowing it to be re-used

Malloc() and free()

University of Exeter

Pattern:

1. Obtain required memory with `malloc()`
2. Use the dynamically allocated memory
3. Release the `malloc`'ed memory with `free()`

Logical Pitfalls:

- You cannot use the memory after it has been freed (p is a dangling pointer)
- You cannot use the memory at p before allocating memory (p is a wild poi
- Do not attempt to free() the memory more than once!

Memory management strategies

In complex programs it can be difficult to trace when and where memory was allocated and when is should be released.

Strategies

- Keep everything on the stack – may run out of stack space
- Use a pre-allocated global array – but how large?
- Allocate and free in the same (wrapper) function:

```
void wrapper()
{
    int *p = malloc(…);
    main_calculation(p);
    free(p);
}
```

- Lazy de-allocation. In short-running programs don't explicitly free – released at program exit

Efficient C: Principles, Practices and Patterns.

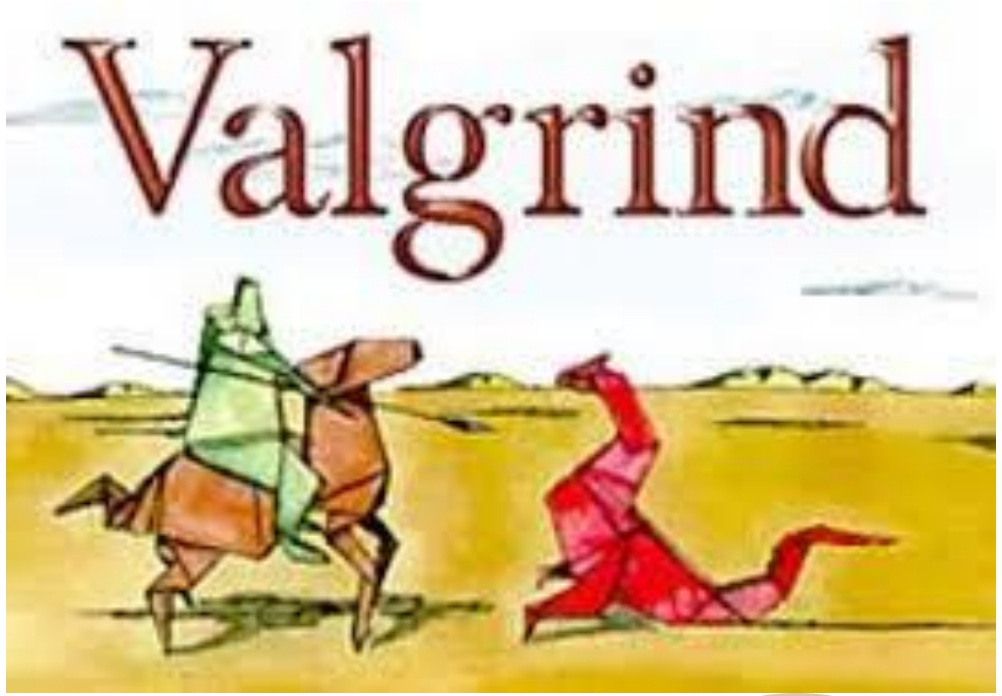
Christopher Preschern
O'Reilly, 2022

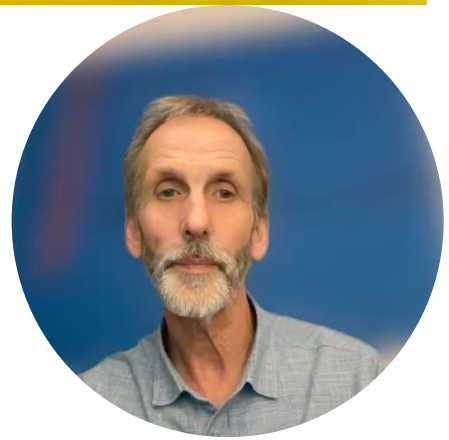
Checking dynamic memory use

University of Exeter

How can you test that all memory is freed properly?

- Address sanitizers (ASAN, UBSAN) instrument code at compile time
- valgrind analyses memory use at run-time

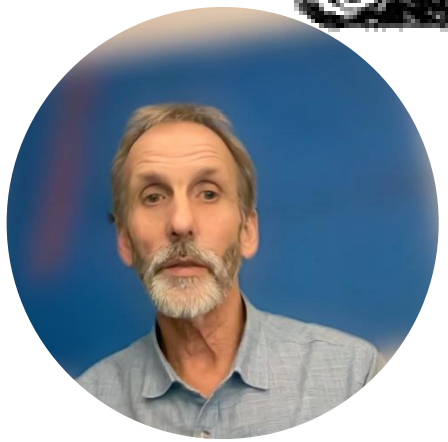




Starting points:
https://github.com/google/sanitizers/wiki/AddressSanitizer
https://www.valgrind.org/docs/manual/QuickStart.html

Dynamic memory with pointers

University of Exeter

- Allows dynamic memory allocation.
- Can lead to intermittent runtime errors.
- Frequently leads to memory leakage.
- Powerful, but can lead to obscure programs.
- Quick!

- Read about malloc internals, calloc, realloc. Wikipedia is a good start
- Read about garbage collection
- C++ and Rust have more sophisticated tools